

Cấu trúc thư mục

Lưu ý: những dòng được bôi vàng là option nâng cao

I. Source tổng

Hãy đưa cả tài liệu và source code vào cùng folder để AI có thêm nhiều tri thức để làm

```
├── docs/
|   ├── srs.md/      # tài liệu đặc tả phần mềm
|   ├── convention.md/  # quy tắc code
|   ├── backlogs/    # chứa backlog
|   |   ├── sprint1/  # chứa các user story, task
|   |   └── sprint2/  # chứa các user story, task
|   ├── DB-erd/      # thiết kế db
|   └── UI/UX style guideline/  # style front end
|
├── code/            # chứa code của dự án
|   ├── front end/
|   └── backend/
|
├── .agent           # chứa các rule, workflow và skill của AI
└── test             # chứa các file test case
```

II. Frontend

Có thể chọn 1 trong 2 cấu trúc dựa vào công nghệ sử dụng

1. Next JS

next-frontend/

```
├── .husky/          # Thư mục chứa các script chặn commit (pre-commit hook) để đảm bảo
code đã chuẩn convention, đáp ứng đúng lint, build không lỗi,... trước khi được đẩy lên
|   └── pre-commit   # Script chạy tsc và eslint trước khi cho phép git commit
```

- |
- |—— public/ # Tài nguyên tĩnh không qua build (favicon, robots.txt, ảnh landing page)
- |
- |—— src/
- | |—— assets/ # Tài nguyên tĩnh đi qua build (images, svg, local fonts)
- | |—— app/ # Next.js App Router (Định nghĩa routing gồm layout, page,... Theo Next)
- | |—— components/ # Chứa các component giao diện
- | | |—— base/ # Các component base tự custom (BaseButton, BaseTable...)
- | | |—— features/ # Component theo nghiệp vụ
- | |—— lib/ # Cấu hình thư viện (Axios client, utils)
- | |—— hooks/ # Custom React Hooks
- | |—— store/ # Global State (Zustand/Redux)
- | |—— types/ # Định nghĩa TypeScript
- | |—— index.css # Global CSS (Cấu hình Tailwind)
- | |—— providers/ # Các provider của react
- | |—— schemas/ # Các validator trên front end (Sử dụng zod)
- | |—— services/ # Các hàm gọi api đến backend
- | |—— utils/ # Chứa các hàm sử dụng chung
- | |—— constants/ # Định nghĩa các hằng số
- |
- |—— eslint.config.mjs # Cấu hình linter khắt khe để đảm bảo chất lượng mã nguồn. Cơ bản sẽ phải kiểm tra những điều sau: eslint-plugin-boundaries (để phân chia ranh giới các phần vào đúng folder), áp dụng quy tắc đặt tên của dự án (kebab-case), không hard code hard text hard mã màu, unusedImports, không lạm dụng any, không sử dụng Magic number
- |—— .prettierrc # Cấu hình format code chuẩn chung cho cả team
- |—— tailwind.config.ts # Cấu hình TailwindCSS, nên ghi đè tailwind bằng ghi đè mã màu
- |—— tsconfig.json # Cấu hình TypeScript (Bật Strict Mode)
- |—— package.json
- |—— Dockerfile, .dockerignore, docker compose # Nếu sử dụng Docker để build
- |—— Sentry # Cấu hình để ghi nhận bug trên product
- |—— .gitignore # Cấu hình những thứ không push lên git

└── .env # Cấu hình biến môi trường, có thể tạo thêm các biến thể như .env.local, .env.development,...

2. React JS

react-frontend/

├── .husky/ # Thư mục chứa các script chặn commit (pre-commit hook) để đảm bảo code đã chuẩn convention, đáp ứng đúng lint, build không lỗi,... trước khi được đẩy lên

| └── pre-commit # Script chạy tsc và eslint trước khi cho phép git commit

|

├── public/ # Tài nguyên tĩnh không qua build (favicon, robots.txt, ảnh landing page)

|

├── src/

| └── assets/ # Tài nguyên tĩnh đi qua build (images, svg, local fonts)

| └── pages/ # Thay thế cho app/ của Next.js (Chứa các trang màn hình)

| | └── Auth/ # Ví dụ: LoginPage.tsx, RegisterPage.tsx

| | └── Dashboard/ # Ví dụ: DashboardPage.tsx

| └── components/ # Chứa các component giao diện

| | └── base/ # Các component base tự custom (BaseButton, BaseTable...)

| | └── features/ # Component theo nghiệp vụ

| └── routes/ # Nơi cấu hình react-router-dom

| | └── index.tsx # Định nghĩa các đường dẫn (path) nối tới các Pages

| └── layouts/ # Chứa bộ khung UI (Header, Sidebar, Footer)

| | └── MainLayout.tsx # Layout cho user đã đăng nhập

| | └── AuthLayout.tsx # Layout cho trang đăng nhập/đăng ký

| └── lib/ # Cấu hình axios, utils chung

| └── hooks/ # Custom React Hooks

| └── store/ # Global State (Zustand/Redux)

| └── types/ # Định nghĩa TypeScript

| └── App.tsx # Component gốc bao bọc toàn bộ ứng dụng (chứa Router Provider)

| └── main.tsx # Điểm neo vào file index.html

| └── styles/ # Cấu hình CSS như theme, global

| └── providers/ # Các provider của react

- | | — schemas/ # Các validator trên front end (Sử dụng zod)
- | | — services/ # Các hàm gọi api đến backend
- | | — utils/ # Chứa các hàm sử dụng chung
- | | — constants/ # Định nghĩa các hằng số
- |
- | — index.html
- | — eslint.config.mjs # Cấu hình linter khắt khe để đảm bảo chất lượng mã nguồn. Cơ bản sẽ phải kiểm tra những điều sau: eslint-plugin-boundaries (để phân chia ranh giới các phần vào đúng folder), áp dụng quy tắc đặt tên của dự án (kebab-case), không hard code hard text hard mã màu, unusedImports, không lạm dụng any, không sử dụng Magic number
- | — .prettierrc # Cấu hình Prettier
- | — tailwind.config.js # Cấu hình TailwindCSS, nên ghi đè tailwind bằng ghi đè mã màu
- | — tsconfig.json # Cấu hình TypeScript (Bật Strict Mode)
- | — vite.config.ts # Cấu hình Vite
- | — Dockerfile, .dockerignore, docker compose # Nếu sử dụng Docker để build
- | — Sentry # Cấu hình để ghi nhận bug trên product
- | — package.json
- | — .gitignore # Cấu hình những thứ không push lên git
- | — .env # Cấu hình biến môi trường, có thể tạo thêm các biến thể như .env.local, .env.development,...

II. Backend Java

Có thể lựa chọn 1 trong 2 kiến trúc Monolith phổ biến sau.

1. Layered Architecture

backend-layered/

- | — src/main/java/com/ioc/internship/
- | | — Application.java # File chạy chính của Spring Boot
- | |
- | | — common/ # Các tiện ích và class cốt lõi dùng chung
- | | | — base/ # Chứa các Base Classes
- | | | | — BaseEntity.java # (@MappedSuperclass có id, created_at, updated_at)

- | | | |—— BaseController.java # (Định nghĩa chung các hàm API phản hồi HTTP)
- | | | |—— BaseService.java # (Interface CRUD dùng Generics <T, ID>)
- | | | |—— BaseServiceImpl.java # (Code thực thi CRUD dùng chung)
- | | |—— constants/ # Hằng số (ErrorCodes, SystemConstants, RegexConstants)
- | | |—— exception/ # Xử lý lỗi toàn hệ thống
- | | | |—— GlobalExceptionHandler.java # (@RestControllerAdvice bắt lỗi tập trung)
- | | | |—— CustomBusinessException.java # Định nghĩa lỗi nghiệp vụ riêng
- | | |—— utils/ # Các hàm phụ trợ (JwtUtils, DateUtils, PasswordEncoder)
- | |
- | |—— config/ # Cấu hình Framework
- | | |—— SecurityConfig.java # Cấu hình phân quyền, chặn API, CORS
- | | |—— OpenApiConfig.java # Cấu hình tài liệu Swagger API
- | | |—— DatabaseConfig.java # Cấu hình kết nối nhiều DB (nếu cần)
- | |
- | |—— controller/ # TẦNG API (Giao tiếp HTTP)
- | | |—— admin/ # Nhóm API cho Admin (ví dụ: AdminUserController)
- | | |—— student/ # Nhóm API cho Sinh viên (ví dụ: StudentProjectController)
- | |
- | |—— dto/ # DATA TRANSFER OBJECT (Chứa object gửi/nhận)
- | | |—— request/ # Dữ liệu Client gửi lên (UserCreateReq, LoginReq) - chứa @Valid
- | | |—— response/ # Dữ liệu trả về (UserDetailRes, PaginatedRes)
- | |
- | |—— entity/ # TẦNG MAP VỚI DATABASE (JPA)
- | | |—— UserEntity.java # Map với bảng users
- | | |—— ProjectEntity.java # Map với bảng projects
- | |
- | |—— repository/ # TẦNG TRUY VẤN DỮ LIỆU
- | | |—— custom/ # Nơi chứa interface gọi Stored Procedure / Native SQL phức tạp
- | | |—— UserRepository.java # Kế thừa JpaRepository cho các câu lệnh đơn giản

```

| |
| |—— service/          # TẦNG NGHIỆP VỤ LỖI
| |—— impl/            # Thư mục bắt buộc chứa code thực thi thật
| |—— UserServiceImpl.java  # Xử lý logic, tính toán, gọi Repository
| |—— UserService.java     # Chỉ chứa Interface (Định nghĩa hành động)
|
|—— unittest/           # Cài thêm unit test để kiểm thử
|—— sonarLint/          # Cài sonarlint để quản lý chất lượng mã nguồn
|—— Dockerfile, .dockerignore, docker compose  # Nếu sử dụng Docker để build
|—— resources/
|—— application.yml      # Cấu hình port, database credentials
|—— text/               # Cấu hình ResourceBundle để lưu text, không hard text vào code
|—— db/migration/       # Nơi viết các thay đổi DB
|—— V1__Init_Tables.sql  # Script tạo bảng CSDL

```

2. Clean Architecture (Thử thách Nâng cao)

backend-clean/

```

|—— src/main/java/com/ioc/internship/
| |—— Application.java
| |
| |—— shared/           # Mã nguồn dùng chéo qua các tầng
| |—— base/             # BaseDomainEntity, BaseUseCase, BaseController
| |—— exception/        # Lỗi kỹ thuật (không dính tới nghiệp vụ)
| |—— utils/            # Các helper thuần Java (không chứa thư viện Spring)
| |
| |—— domain/           # TẦNG LỖI
| |—— entity/           # Plain Java Model
| |—— enums/            # Các trạng thái nghiệp vụ (UserRole, ProjectStatus)
| |—— exception/        # Lỗi nghiệp vụ (UserNotFoundException,
InvalidScoreException)
| |—— repository/       # CHỈ CÓ INTERFACE (IUserRepository)

```

		└── application/	# TẦNG NGHIỆP VỤ (USE CASE)
		└── dto/	# Model truyền vào Use Case (Input/Output Boundary)
		└── usecase/	# Interface quy định hành động (Ví dụ: CreateUserUseCase)
		└── service/	# Class implement UseCase. Nó sẽ được "tiêm" IUserRepository vào để làm việc.
		└── infrastructure/	# TẦNG HẠ TẦNG (KẾT NỐI VỚI THẾ GIỚI BÊN NGOÀI & DB)
		└── config/	# Spring Security, Bean Configuration, Kafka/Redis config
		└── persistence/	# Phần giao tiếp Database
		└── entity/	# JPA Entity (Chứa các Annotation @Entity, @Table)
		└── repository/	# Spring Data JpaRepository
		└── adapter/	# MẮT XÍCH QUAN TRỌNG: Class implement IUserRepository (ở tầng Domain), gọi xuống JpaRepository để lưu dữ liệu, đồng thời map JPA Entity <-> Domain Entity.
		└── presentation/	# TẦNG HIỂN THỊ (GIAO TIẾP HTTP)
		└── controller/	# Rest Controllers (Nhận HTTP, gọi xuống tầng Application/UseCase)
		└── payload/	# HTTP Request Body, JSON Response
		└── exception/	# @RestControllerAdvice để map lỗi Domain sang HTTP Status Code (400, 404, 500)
		└── unittest/	# Cài thêm unit test để kiểm thử
		└── sonarLint/	# Cài sonarlint để quản lý chất lượng mã nguồn
		└── Dockerfile, .dockerignore, docker compose	# Nếu sử dụng Docker để build
		└── resources/	
		└── application.yml	
		└── text/	# Cấu hình ResourceBundle để lưu text, không hard text vào code
		└── db/migration/	# Nơi viết các thay đổi DB

III. Backend NestJS

backend-nestjs/

├── .husky/ # Thư mục chứa các script chặn commit (pre-commit hook) để đảm bảo code đã chuẩn convention, đáp ứng đúng lint, build không lỗi,... trước khi được đẩy lên

├── src/

| ├── main.ts # Entry point (Khởi tạo app, setup Global Pipes/Filters/Swagger)

| ├── app.module.ts # Root Module (Gắn kết mọi thứ lại với nhau)

| |

| ├── common/ # [TẦNG DÙNG CHUNG] (Cross-cutting concerns)

| | ├── base/ # [LỰA CHỌN NÂNG CAO] Các class chung để tái sử dụng

| | | ├── base.dto.ts # PaginationDto, ApiResponseDto

| | | ├── base.service.ts # Abstract Class định nghĩa sẵn CRUD bằng Generics

| | | └── base.controller.ts # Abstract Controller định nghĩa sẵn GET, POST, PUT, DELETE

| | ├── constants/ # Hằng số (Messages, ErrorCodes)

| | ├── decorators/ # Custom Decorators (VD: @CurrentUser(), @Roles())

| | ├── exceptions/ # Custom Exceptions (Business exceptions)

| | ├── filters/ # Bắt lỗi toàn cục (AllExceptionsFilter)

| | ├── guards/ # Chặn cổng bảo mật (JwtAuthGuard, RolesGuard)

| | ├── interceptors/ # Ghi log request, biến đổi Response cho đồng nhất

| | ├── i18n/ # chứa các text đa ngôn ngữ

| | └── utils/ # Các hàm tiện ích (BcryptHelper, DateHelper)

| |

| ├── config/ # Cấu hình môi trường (Load từ .env)

| | └── app.config.ts

| |

| ├── database/ # Cấu hình kết nối DB

| | └── prisma.service.ts # Khởi tạo Prisma Client để chọn xuống DB

| |

| └── modules/ # 📦 [TẦNG NGHIỆP VỤ LỖI] - Gói theo Feature

| |

| └── auth/ # Module Đăng nhập / Phân quyền

- | | |—— dto/ # login.dto.ts, register.dto.ts
- | | |—— strategies/ # jwt.strategy.ts (Xác thực token)
- | | |—— auth.controller.ts
- | | |—— auth.service.ts
- | | |—— auth.module.ts # Định nghĩa các Provider và Controller của Auth
- | |
- | |—— students/ # Module Quản lý Sinh viên
- | | |—— dto/ # create-student.dto.ts, update-student.dto.ts
- | | |—— student.controller.ts # [Kế thừa BaseController]
- | | |—— student.service.ts # [Kế thừa BaseService]
- | | |—— student.module.ts
- | |
- | |—— projects/ # Module Quản lý Dự án (Cấu trúc tương tự)
- |
- |—— prisma/ # TẦNG DATABASE (Prisma ORM)
- | |—— schema.prisma # Khai báo cấu trúc bảng CSDL (Entities)
- | |—— migrations/ # Chứa lịch sử thay đổi DB
- |
- |—— .env # Biến môi trường
- |—— nest-cli.json # Cấu hình của Nest CLI
- |—— package.json
- |—— eslint.config.mjs # Khuyến dùng bộ ESLint chặt chẽ như Frontend
- |—— .prettierrc
- |—— Dockerfile, .dockerignore, docker compose # Nếu sử dụng Docker để build
- |—— .gitignore # Cấu hình những thứ không push lên git
- |—— tsconfig.json # (Bật strict mode)